

Hiku: the mathematics of a subtraction-jump puzzle

Espen Schulstad
Rubberduck AS, Norway

October 2026

Abstract

Hiku is a solitaire puzzle on a square grid. Each number jumps exactly as many squares as it shows, along its row or column, and must land on another number; the smaller is subtracted from the larger and the difference stays where the jump landed. The goal is to make every number vanish. We give a precise model of the game and prove its basic structure: the sum of the numbers keeps its parity, the occupied squares only ever shrink, and a position splits into independent groups, the connected components of a bipartite graph on rows and columns, which never merge again. These facts yield cheap certificates of loss that a player can check at a glance, and a solver that is an order of magnitude faster than plain search with memoisation. Boards whose numbers are all ones are solvable exactly when their squares admit a perfect matching in the grid graph, so this case is decidable in polynomial time; the general problem is in NP and we conjecture it is NP-complete. We then turn to difficulty. We define exact measures computed from the full game graph—the share of safe moves, the number of tight decisions, how long a trap stays hidden, and the win rate of random and greedy play—and a new measure, the *refutation depth* of a trap: how far a player must look ahead before every continuation runs into a loss visible at a glance. Measured exactly on a stratified sample of 160 of the 4384 daily boards shipped with the game, refutation depth separates the four difficulty tiers sharply, and a player who looks a fixed number of moves ahead wins the easiest boards almost always but the expert boards almost never.

1 Introduction

Hiku¹ is a number puzzle in the family of jump-and-remove solitaire games, of which peg solitaire is the best known [1, 4]. It differs from peg solitaire in two ways that shape everything below. First, the length of a jump is not fixed: a number jumps exactly as far as its value, over anything in between. Second, a jump does not remove the number jumped over; it combines the jumper with the number it lands on, by subtraction. A number therefore changes its own reach as the game goes on, and interactions are not local: a 6 on a 7×7 board reaches across the whole board.

The game was designed to be learned in a minute and to reward thinking ahead. Players are told three rules and three tricks, and each board is guaranteed to be solvable. This paper does two things. It develops the mathematics of the game far enough to explain *why* the tricks work and to build a fast exact solver (sections 3, 4 and 8). And it asks what makes a board hard, proposing exact measures of difficulty computed from the complete game graph of each board and testing them on the boards the game ships with (sections 5 and 6).

Contributions.

- A formal model of the game (section 2).

¹Available on the App Store for iPhone. The name is the Japanese verb 引く, “to pull”, which is also the word for subtraction: $5 \text{ 引 } 2 \text{ は } 3$, “five minus two is three”.

- Three invariants—parity of the sum, shrinking occupancy, and a non-increasing maximum—and a decomposition theorem: a position is solvable if and only if each connected component of its *row-column graph* is (section 3). Three local certificates of loss follow.
- A polynomial-time characterisation of all-ones boards via perfect matchings, a proof that the general problem is in NP, and a conjecture that it is NP-complete (section 4).
- Exact difficulty measures and the notion of *refutation depth* of a trap, measured over the boards in the game; a model of a player with bounded look-ahead and its win rates per tier (sections 5 and 6).
- A solver that combines a 256-bit packed position, a transposition table, parity pruning and component decomposition, eight to twelve times faster than the one it replaced (section 8).

2 Rules

Definition 1 (Position). A *position* on an $N \times N$ board is a map p from the squares (r, c) , $0 \leq r, c < N$, to the non-negative integers. A square s is *occupied* if $p(s) > 0$, and $O(p)$ is the set of occupied squares. We write $n(p) = |O(p)|$ and $S(p) = \sum_s p(s)$.

Definition 2 (Move). Let $s = (r, c)$ be occupied with value $v = p(s)$ and let d be one of the four unit directions $(0, \pm 1), (\pm 1, 0)$. If $t = s + v d$ lies on the board and is occupied, the *move* $s \rightarrow t$ produces the position p' with

$$p'(s) = 0, \quad p'(t) = |p(s) - p(t)|, \quad p'(u) = p(u) \text{ otherwise.}$$

A number thus jumps exactly its value, horizontally or vertically, over whatever lies between, and must land on a number. If the two are equal both vanish; otherwise the difference remains on the landing square. A position with no occupied square is *clear*. A position is *solvable* if some sequence of moves leads from it to the clear position, and *lost* otherwise; a lost position with no legal move is *stuck*. The puzzle is: given a solvable starting position, find such a sequence. Figure 1 shows a small board.

In the game N ranges from 3 to 8 and values from 1 to 7; a number of value $v \geq N$ can never move, though it can still be landed on. The rules are invariant under the eight symmetries of the square, which we use to count boards up to symmetry.

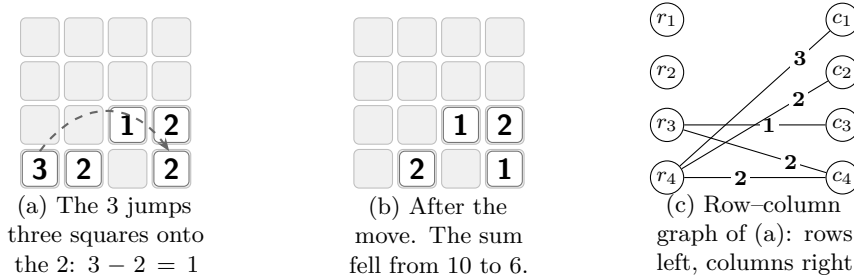


Figure 1: Lesson board “Even numbers”. Each number is an edge between its row and its column in the row-column graph (c); the five numbers form one component, with an even sum. After the move in (a), the 3’s edge r_4c_1 is gone.

3 Structure

3.1 Invariants

Lemma 3 (Occupancy). *If $p \rightarrow p'$ is a move, then $O(p') \subsetneq O(p)$, and $n(p') = n(p) - 1$ if the two numbers differ and $n(p) - 2$ if they are equal.*

Proof. The jumping square becomes empty and the landing square keeps a number only if the difference is non-zero. No other square changes; in particular no empty square is ever filled. $\square$

Corollary 4 (Length of play). *Every game ends after at most $n - 1$ moves. A solution from a position with n numbers has exactly $n - e$ moves, where $e \geq 1$ is the number of moves between equal numbers; the last move of every solution is between equal numbers.*

Proof. Each move removes one or two numbers, so a solution of m moves with e equal moves removes $m + e = n$ numbers. The last move leaves the board clear, so it removes two numbers, and those were equal. $\square$

Lemma 5 (Sum). *If $p \rightarrow p'$ combines a and b , then $S(p') = S(p) - 2 \min(a, b)$. In particular the parity of S never changes, and the largest value on the board never increases.*

Proof. The two numbers a, b are replaced by $|a - b| = a + b - 2 \min(a, b)$. And $|a - b| < \max(a, b)$ for positive a, b . $\square$

Corollary 6. *A position with an odd sum is lost. In any solution the minima of the combined pairs add up to exactly $S/2$.*

Note that the value on a single square is *not* monotone: a 5 landing on a 1 leaves a 4 where the 1 was. Only the maximum over the board is monotone: it never increases.

3.2 The row-column graph and its components

A move combines two numbers that share a row or a column. This suggests looking at the board as a graph on its lines.

Definition 7 (Row-column graph). The *row-column graph* $G(p)$ of a position is the bipartite graph with one vertex for every row and every column, and one edge $\{r, c\}$, labelled $p(r, c)$, for every occupied square (r, c) . A *group* of p is the set of occupied squares belonging to one connected component of $G(p)$.

Two numbers are in the same group if one can get from one to the other by steps between numbers sharing a row or a column. A move $s \rightarrow t$ acts on two edges with a common endpoint and, by lemma 3, deletes one or both of them.

Lemma 8 (Groups only split). *If $p \rightarrow p'$ is a move, $G(p')$ is a subgraph of $G(p)$ with the same labels on the surviving edges except the landing edge. Hence every group of p' is contained in a group of p : groups can split but never merge.*

Theorem 9 (Decomposition). *Let $p_1, \dots, p_k$ be the restrictions of p to its groups. Then p is solvable if and only if every p_i is solvable. Moreover, the solutions of p are exactly the interleavings of solutions of the p_i .*

Proof. A move involves two squares sharing a line, hence two squares of the same group, and changes nothing outside them. So a move of p_i is a move of p with the same effect on the squares of group i , and moves in different groups commute. Conversely, by lemma 8 the squares a move of p touches, at any time, lie inside one of the original groups, and the moves of a sequence that touch group i form, in order, a legal sequence for p_i with the same effect on it. A sequence clears p exactly when it clears every group. $\square$

Theorem 9 turns a question about one board into questions about smaller ones. Its corollaries are the certificates of loss a player can check without any search.

Corollary 10 (Glance rules). *A position is lost if it has a group*

- (i) with a single number (a number sharing no row or column with any other);
- (ii) whose sum is odd; or
- (iii) with exactly two numbers a and b , unless $a = b$ and they are exactly a squares apart.

Proof. (i) The number has no partner, now or later. (ii) By lemma 5 applied to the group alone. (iii) Two numbers in one group share a line. Any move between them leaves $|a - b|$, which is a lone number unless $a = b$; and the move exists only if the distance equals the jumping value. $\square$

We call a position *flagged* if one of the glance rules applies to it, or it is stuck. Flagged positions are lost; the converse fails, and how badly it fails is the subject of section 6.

3.3 Playing backwards

The game can be run in reverse, which is how solvable boards are produced.

Proposition 11 (Reverse moves). *Let p' be a position, s an empty square and t a square $h \geq 1$ steps from s along a line. Put $w = p'(t) \geq 0$ and choose $b \geq 1$ with $|h - b| = w$, that is $b = h + w$, or $b = h - w$ if $w < h$. The position p equal to p' except $p(s) = h$ and $p(t) = b$ has the move $s \rightarrow t$ leading to p' . Every solvable position arises from the clear position by a sequence of such reverse moves.*

The app's generator starts from the clear board and applies random reverse moves until it reaches a chosen number of numbers; every board it produces is solvable by construction. Boards for publication are then improved by local search on the difficulty measures of section 5.

4 Complexity

Let HIKU be the decision problem: given an $N \times N$ position with values written in binary, is it solvable?

Proposition 12. *HIKU is in NP.*

Proof. By corollary 4 a solution has fewer than $n \leq N^2$ moves. A certificate lists them as pairs of squares, and each move is verified by comparing a distance with a value and subtracting two values, in time polynomial in the length of the input. $\square$

The number of positions reachable from a start with n numbers of value at most V is at most $(V + 1)^n$, since each occupied square either empties or holds a value of at most V (lemma 5). Plain search is therefore exponential, and the question is whether that is necessary.

4.1 All ones: a polynomial case

Theorem 13. *A position whose numbers are all 1 is solvable if and only if its occupied squares have a perfect matching in the grid graph (squares adjacent horizontally or vertically). Hence this case is decidable in polynomial time.*

Proof. A 1 can only jump onto an adjacent 1, and $1 - 1 = 0$, so every move removes two adjacent numbers and leaves every other value at 1. A solution therefore partitions the occupied squares into adjacent pairs, which is a perfect matching. Conversely, given a perfect matching, play its pairs one at a time: the squares of different pairs are disjoint, so each pair is still present and adjacent when its turn comes. Perfect matching in a bipartite graph is decided in $O(E\sqrt{V})$ time by the Hopcroft–Karp algorithm [2]. $\square$

We checked theorem 13 against the exact solver on 10 000 random all-ones boards of sizes 4 to 7: the two agreed on every board.

4.2 The general case

With larger values the argument breaks down in two places at once. A move no longer removes both numbers, so a number can take part in several moves; and a surviving number changes its reach, so the set of possible partners changes as the game goes on. Peg solitaire, which has only the first feature, is NP-complete [4], as are many single-player puzzles [3], among them Sudoku [5].

Conjecture 14. *HIKU is NP-complete, already for values bounded by a constant.*

We do not have a proof. Any reduction has to respect theorem 9: gadgets that are meant to interact must be connected in the row-column graph, and a wire that has been used up may split the board into parts that can then be solved independently. We leave the question open.

5 Difficulty

Every board in the game is solvable, so difficulty is not about whether a solution exists but about how hard it is to find. Since the boards are small, all our measures are computed exactly from the full game graph of each board: every position reachable from the start, with its solvability. We call a move *safe* if it leads to a solvable position and a *trap* otherwise.

5.1 Measures

Careful play is a player who never makes a trap and picks uniformly among the safe moves. The following measures are expectations over careful play or over uniformly random play.

Safe share. The share of legal moves that are safe, averaged over the positions of careful play.

Tight decisions. The expected number of positions in careful play where at most one legal move in three is safe.

Hidden depth. After a trap, the expected number of further moves a random player makes before being stuck: how long the mistake stays out of sight.

Random wins. The probability that uniformly random play clears the board.

Greedy. Whether the rule “combine equal numbers first, otherwise use the largest number” clears the board.

A board is assigned to the hardest tier whose every threshold it meets (table 1). The thresholds were chosen so that easy boards are forgiving and a single trap may be all there is, while expert boards have a trap behind most moves and one that stays hidden for several moves.

Tier	Greedy	Random wins	Safe share	Hidden depth	Tight decisions
Easy	may clear	$\leq 20\%$	$\leq 65\%$	≥ 1.5	≥ 1
Medium	fails	$\leq 5\%$	$\leq 50\%$	≥ 2.5	≥ 2
Hard	fails	$\leq 1\%$	$\leq 40\%$	≥ 3.5	≥ 3
Expert	fails	$\leq 0.2\%$	$\leq 30\%$	≥ 4.5	≥ 4

Table 1: Thresholds for the four tiers.

5.2 The board corpus

The game has 200 levels: 50 free, the first nine of them lessons, and 150 more set aside for a later purchase. It also ships a library of 4384 daily boards, 1096 per tier, enough for three years of four boards a day. They were found by hill climbing: starting from generated boards, change one number at a time and keep the change if the board is still solvable and no easier by the measures above. Duplicates up to symmetry are removed. Table 2 summarises them. Easy daily boards are 4×4 and 5×5 with about ten numbers and values up to 4; expert boards are 6×6 to 8×8 with about seventeen numbers and values up to 7.

Boards	Count	Solutions	Safe	Tight	Hidden	Random wins	Greedy
Daily easy	1096	6 720	58%	1.5	2.3	3.0%	3%
Daily medium	1096	90 092	46%	4.0	3.0	0.086%	0%
Daily hard	1096	665 280	37%	8.4	3.8	0.001%	0%
Daily expert	1096	92 160	27%	13.9	4.7	< 0.001%	0%
Levels 10–14, easy	5	80	59%	1.8	1.8	4.3%	20%
Levels 15–20, easy	6	246 512	60%	1.4	2.7	3.2%	17%
Levels 21–35, medium	15	55 520	46%	4.2	2.8	0.14%	0%
Levels 36–43, hard	8	3 344	34%	8.8	3.5	0.001%	0%
Levels 44–50, hard	7	237 120	37%	7.8	3.7	0.001%	0%
Levels 51–100, hard	50	665 280	37%	8.5	3.8	0.001%	0%
Levels 101–150, expert	50	4 860	27%	12.6	4.6	< 0.001%	0%
Levels 151–200, expert	50	230 074	27%	14.0	4.7	< 0.001%	0%

Table 2: The board corpus: the daily library and levels 10–200 (levels 51–200 are not yet released). *Solutions* is the median number of move orders that clear a board; the other columns are means.

The median number of solutions shows that difficulty is not scarcity: a hard daily board typically has hundreds of thousands of solutions. What makes it hard is that the traps outnumber the safe moves at almost every step and stay hidden for several moves.

6 How far must a player look?

The measures above describe the game graph. They do not say what a player has to *do* to avoid a trap. Human players do not search; they apply rules they can see at a glance, like those of corollary 10, and they look a few moves ahead. We make this precise.

Definition 15 (Refutation depth). For a lost position q , let $\rho(q) = 0$ if q is flagged (stuck, or ruled out by corollary 10), and otherwise

$$\rho(q) = 1 + \max_{q \rightarrow q'} \rho(q').$$

The refutation depth of a trap is ρ of the position it leads to.

Every continuation of a lost position is lost, so ρ is well defined, and it is at most the number of moves left. It is the depth of the smallest tree of look-ahead in which every branch ends in a position the player can see to be lost: a trap of depth 0 is visible immediately after the move, a trap of depth 2 only to a player who checks every reply and every reply to that. It is a demanding notion—the player must refute *all* lines—which is exactly what it takes to be sure.

Definition 16 (Player with vision k). A player with vision k never makes a move whose resulting position is lost with refutation depth less than k , and picks uniformly among the

other moves (or among all moves, if none is left). Vision 0 is random play; vision 1 is a player who applies the glance rules after each candidate move.

Table 3 shows the distribution of refutation depths over all traps reachable from solvable positions, and fig. 2 the probability that a player with vision k clears a board, both exactly computed over 40 boards per tier drawn evenly from the daily library.

Tier	Solvable positions	Traps per position	Refutation depth of traps					
			0	1	2	3	4	≥ 5
Easy	132	3.2	35%	7%	14%	16%	13%	15%
Medium	306	4.8	19%	3%	8%	13%	15%	43%
Hard	415	6.2	16%	1%	4%	7%	10%	63%
Expert	177	8.8	9%	1%	2%	3%	5%	80%

Table 3: Traps by refutation depth, over every trap out of every solvable position reachable from the start, on 40 daily boards per tier. *Solvable positions* is the mean per board; *traps per position* the mean number of traps available in a solvable position.

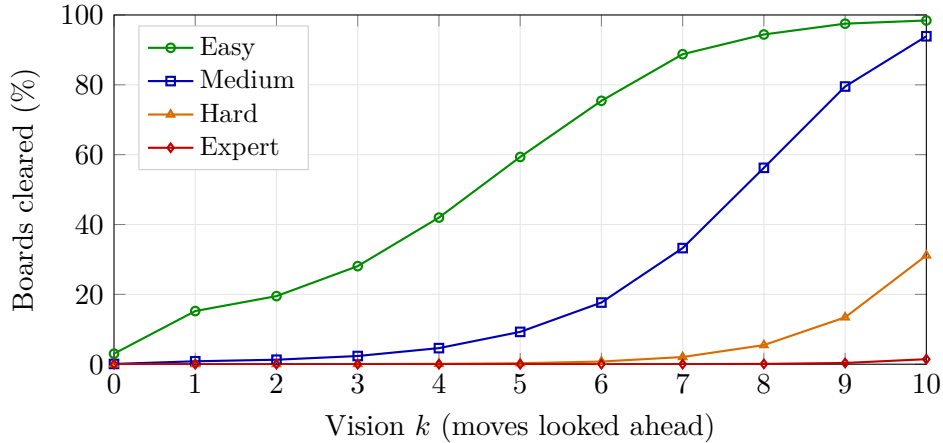


Figure 2: Probability that a player with vision k clears a board, averaged over 40 daily boards per tier. Vision 0 is random play; vision 1 applies the glance rules to each candidate move.

The tiers separate cleanly. On easy boards a third of the traps (35%) are visible at a glance right after the move, and only 15% need five or more moves of look-ahead; on expert boards it is 9% and 80%. Traps also become more plentiful: a solvable position offers 3.2 traps on average on easy boards and 8.8 on expert boards, while the number of solvable positions stays small—a few hundred per board. The deepest trap we found needs 16 moves of look-ahead to refute.

The win rates in fig. 2 show what this means for a player. The glance rules alone (vision 1) raise the chance of clearing an easy board from 3% to 15%, and a player who looks seven moves ahead clears 89% of them. Medium boards need about eight moves of look-ahead for an even chance (56% at $k = 8$), and hard boards stay below one in three even at $k = 10$ (31%). Expert boards are cleared by a player with vision 10 only 1.4% of the time. The model is pessimistic about people in one respect—it credits the player with no reasoning beyond the glance rules and exhaustive look-ahead, while a strong player also uses the techniques of section 7—and optimistic in another, since looking ten moves ahead through every reply is far beyond what anyone does in their head. What it captures is the shape: each tier demands a few more moves of foresight than the one below it.

7 Techniques for players

The lessons in the game teach three tricks. Each has a precise counterpart above.

One way out. *“A number that can reach only one other must take it. Don’t let anything else take it first.”* Let x on square s have exactly one occupied landing square t . As long as nothing lands on x , its value is unchanged, so its landing squares are fixed, and the empty ones stay empty (lemma 3). If nothing will ever land on x —the typical case for a large number, or one no other number is in line with—then every solution contains the move $s \rightarrow t$, and square t must still be occupied when it is played. The number on t may change on the way (if something lands on it, a difference stays there), but it must not jump away or vanish first. That is what “don’t let anything else take it” means: the square x depends on must not be emptied.

Even numbers. *“Numbers that can only reach each other vanish only if they add up to an even number.”* This is corollary 10(ii), stated for the case players see most: a group that has split off.

Two moves ahead. *“Before you jump, see what has to happen next.”* This is vision (section 6). The results there say that the glance rules alone win few boards, and that the depth needed grows with the tier.

Further techniques that follow from the structure:

- *Don’t strand a number.* Before a move, check that no number is left without a row- or column-mate (corollary 10(i)). Moving a number out of a line can strand the one left behind.
- *Count per group, not per board.* After a split, each group has to have an even sum on its own.
- *Plan the ending.* The last move always combines two equal numbers exactly their value apart (corollary 4); with few numbers left, work backwards from a pair that could be last.
- *Big numbers are anchors.* The largest value never grows (lemma 5); a large number with no landing square has to be worn down by others landing on it, and each landing by a smaller x reduces it by x .
- *Pay for the sum.* Every move removes twice the smaller number from the total, and the minima of all moves add up to exactly half the starting sum (corollary 6).

8 A fast solver

The app checks after every move whether the board can still be cleared, and warns the player if not. This check must be fast on a phone. The solver is a depth-first search over positions with a transposition table (a hash map from position to its settled answer), a budget on the number of new positions it may visit, and three ingredients:

1. *Packed positions.* A position of up to 8×8 squares with values below 16 fits in four 64-bit words, four bits a square, so hashing and copying do not allocate.
2. *Decomposition* (theorem 9). The groups are found with bitmasks: starting from the lowest occupied square, repeatedly add every occupied square in a row or column of the group. If there is more than one group, each is solved separately and the answers combined; the table then stores small groups that recur across many positions.

3. *Glance rules* (corollary 10(i)–(ii)): a lone number ends the search at once, and whenever a position splits, a group with one number or an odd sum settles it as lost without search.

Only settled answers are stored; a search that runs out of budget returns “unknown” and the app shows no warning. Table 4 compares it with the earlier solver, which had the same search and table over positions stored as arrays, on random games played on every eighth daily board. In the app the check is also run off the main thread when it does not settle within 2 000 positions, so a move is never delayed by it.

Tier	Checks	Earlier solver		Current solver		Speed-up
		Mean	Worst	Mean	Worst	
Easy	749	0.10 ms	8 ms	0.01 ms	1 ms	8×
Medium	935	0.91 ms	207 ms	0.10 ms	21 ms	9×
Hard	1140	2.81 ms	294 ms	0.26 ms	27 ms	11×
Expert	1376	19.66 ms	1162 ms	1.60 ms	96 ms	12×

Table 4: Time to check whether a position can still be cleared, after every move of random games on every eighth daily board, with a budget of 150 000 positions (Apple M1 Max, one core). With that budget the earlier solver gave up on 3 expert positions; the current one settled all.

9 Random boards

How special are the curated boards? Table 5 shows how often a random board can be cleared, for numbers placed uniformly on distinct squares with values uniform in $1, \dots, N - 1$, and how much of the unsolvability the glance rules explain at the start, before any move.

Board	Numbers	Solvable	Odd sum	Lost positions flagged at start
4×4	4	1.65%	50%	85%
4×4	8	5.80%	50%	57%
5×5	4	0.35%	50%	90%
5×5	8	0.60%	51%	69%
5×5	12	2.40%	49%	51%
6×6	4	0.15%	50%	93%
6×6	8	0.05%	51%	82%
6×6	12	0.05%	49%	56%
6×6	16	1.10%	48%	49%
7×7	4	0.05%	49%	96%
7×7	8	0.00%	50%	90%
7×7	12	0.00%	50%	71%
7×7	16	0.00%	50%	55%

Table 5: Random boards at selected densities, 2000 per row. The totals in the text cover all 22 combinations of size (4 to 7) and even numbers of numbers from 4 up to 16 or half the squares. Columns: the share that can be cleared, the share with an odd sum, and of those that cannot be cleared, the share already ruled out by the glance rules before any move.

Random boards are almost never solvable: 352 of 44 000 over all sizes and densities, and 1 of 14 000 on 7×7 . Half fail by parity alone. With few numbers most of the rest are ruled out at a glance, because numbers are rarely in line with each other; with more numbers the board becomes one group and the glance rules explain little beyond parity. Solvable boards are thus

rare objects, which is why the generator builds them backwards (proposition 11) rather than sampling and checking.

10 Conclusion

Hiku has a simple rule and a rich structure. Its sum keeps its parity, its board only empties, and it falls apart into independent groups that never rejoin; these facts give players rules they can check at a glance and give the solver an order of magnitude. They are not enough on their own: on the hardest boards the traps are deep, and a player has to look many moves ahead to be sure of a move. Whether the general problem is NP-complete remains open.

Acknowledgements. The analysis, the solver and the experiments were developed with the help of Claude (Anthropic). The code of the experiments is in `Tools/paper-experiments.swift` in the game’s repository.

References

- [1] J. D. Beasley. *The Ins and Outs of Peg Solitaire*. Oxford University Press, 1985.
- [2] J. E. Hopcroft and R. M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973. doi:10.1137/0202019
- [3] G. Kendall, A. Parkes and K. Spoerer. A survey of NP-complete puzzles. *ICGA Journal*, 31(1):13–34, 2008. doi:10.3233/ICG-2008-31103
- [4] R. Uehara and S. Iwata. Generalized Hi-Q is NP-complete. *Transactions of the IEICE*, E73-E(2):270–273, 1990.
- [5] T. Yato and T. Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E86-A(5):1052–1060, 2003.